Kaching

Point-of-Sale Application

Document purpose: Define the system-wide architecture, technology choices, interfaces, data strategy, security controls, deployment topology, and design conventions that all Kaching features must follow.

Field	Value
Document	System-Level Software Design Specification
Version	0.1
Status	Sample Draft for Editing
Date	17 August 2026
Prepared by	[Student Name]
Product	Kaching

Design baseline

- Current SRS containing FR-001 to FR-038, BR-001 to BR-020, and NFR-001 to NFR-011.
- Feature Inventory containing Feature-A to Feature-R.
- Approved additions for administration, barcode-based product entry, global catalog pricing, capacity, recovery, connectivity, and English-only user interface.

Document Control

Version	Date	Author	Summary
0.1	17 Aug 2026	[Student Name]	Initial system-level design based on the approved SRS and clarification decisions.

Contents

1. Introduction

1.1 Purpose

This SDS defines the system-level design for Kaching, a multi-store point-of-sale application. It translates the approved requirements and feature inventory into a coherent technical architecture. It intentionally stops short of screen-by-screen, class-level, and endpoint-level feature specifications, which belong in later feature design documents.

1.2 Scope

Kaching supports counter checkout, order-level discounts, Thai VAT-inclusive totals, cash and credit-card payments, durable sale recording, receipt printing, asynchronous inventory updates, daily sales reporting, product and price maintenance, operational exception monitoring, auditing, and administration of users, stores, and terminals.

1.3 Exclusions

- Returns, refunds, and refund receipts.
- Customer Service Counter integration.
- Split payments.
- Synchronous inventory-availability checking.
- Offline sale processing when a terminal cannot reach the Kaching backend.

1.4 Intended Audience

The intended readers are software engineers, architects, database designers, testers, DevOps engineers, security reviewers, instructors, and future feature-design authors.

1.5 Terms

Term	Meaning
PSP	Payment Service Provider, represented by K-Bank in this design.
ECR	Electronic cash register protocol used to send a payment amount to a card terminal and receive a result.
Outbox	A database record written in the same transaction as a completed sale and later published asynchronously.
Peripheral Gateway	A terminal-side Node.js service that connects the browser workflow to the card machine and receipt printer.
Business date	The calendar date in Asia/Bangkok used for store reporting.

2. Architectural Drivers

2.1 Business and Design Goals

- Keep checkout fast and simple for store staff.
- Prevent duplicate card charges when a payment outcome is uncertain.
- Guarantee that every completed sale has a durable inventory-update record.
- Allow inventory processing to continue asynchronously when the Inventory Service is unavailable.
- Support multiple stores and terminals from one centrally managed application.
- Use a design that is realistic for production while remaining understandable and maintainable as a student project.

2.2 Fixed Constraints and Assumptions

Area	Decision
Frontend	React, TypeScript, and Vite.
Backend	Node.js, Express, and TypeScript.
Persistence	PostgreSQL accessed through Prisma ORM.
Hosting	Centralized Google Cloud deployment using containerized services and Cloud SQL.
Messaging	RabbitMQ with durable queues and publisher confirmations.
Connectivity	A terminal must reach the Kaching backend to start or complete a sale.
Integrations	Production-style interfaces for K-Bank, the card machine, receipt printer, and Inventory Service.
Language	English-only user interface; Thai currency, VAT, and Asia/Bangkok business time.
Catalog	One global product catalog and one global selling price per product.

2.3 Quality and Capacity Targets

Quality area	Target
Scale	20 stores, 10 active terminals per store, and 200 concurrent users.
Throughput	At least 20 completed sales per second across the organization.
Checkout response	95 percent of non-external checkout actions complete within 2 seconds.
Availability	99.5 percent monthly availability for the Kaching backend.
Recovery point	RPO of 15 minutes.
Recovery time	RTO of 4 hours.

Quality area	Target
Display	Desktop and touchscreen friendly at 1280 x 720 or higher.
Accessibility	WCAG 2.1 AA color contrast and keyboard-operable workflows.

3. System Context

Kaching is the system of record for checkout transactions and their payment status. It does not own inventory balances or cardholder data. The Inventory Service owns stock balances, while K-Bank and the card machine own sensitive payment processing.

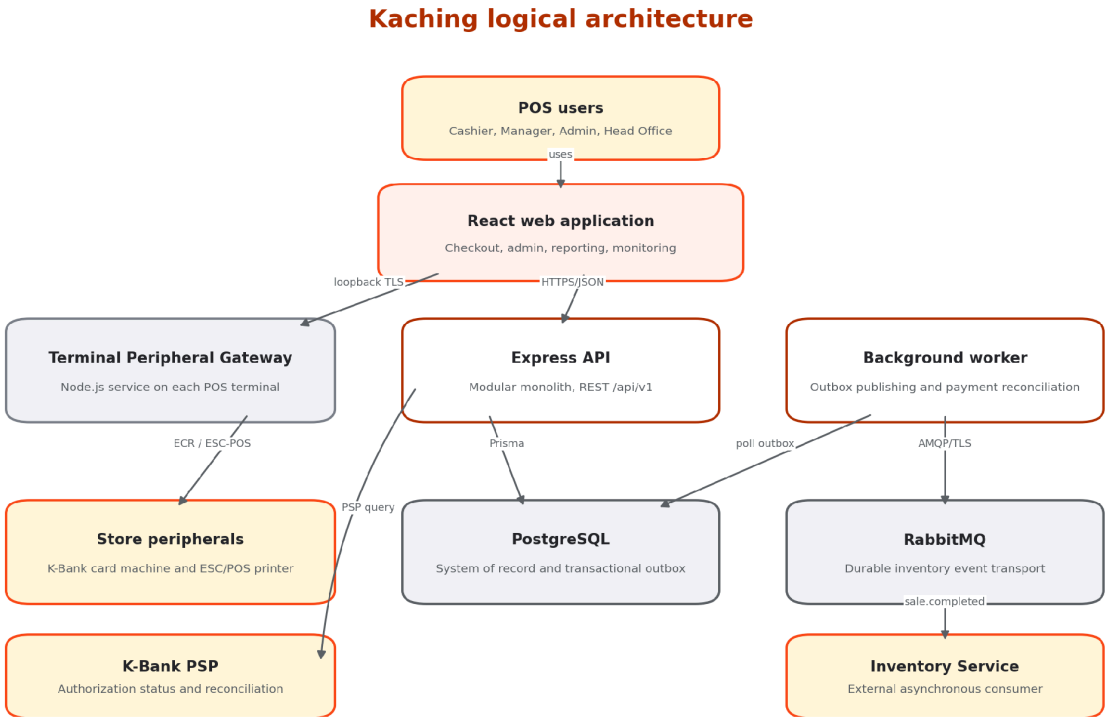

Figure 1. Kaching logical architecture

3.1 External Actors and Systems

Actor or system	Responsibility	Interaction
Cashier	Operates checkout and payment workflows.	React web application
Manager	Views assigned-store summaries and maintains authorized product data.	React web application
Administrator	Maintains users, roles, stores, terminals, and cross-store access.	React web application
Head-Office Manager	Views selected-store and consolidated reports.	React web application

Actor or system	Responsibility	Interaction
K-Bank card machine and PSP	Authorizes card payment and supports transaction-status queries.	Peripheral Gateway and PSP adapter
Receipt printer	Prints sales receipts.	Peripheral Gateway
Inventory Service	Consumes completed-sale messages and adjusts stock.	RabbitMQ

4. Architecture Overview

4.1 Architectural Style

Kaching uses a modular monolith for the business API, plus a separately deployed background worker and terminal-side Peripheral Gateway. This avoids premature microservice complexity while preserving clear module boundaries and independent scaling for asynchronous work.

Primary design rule: PostgreSQL is the source of truth. External calls must never be used as substitutes for durable local state, and a completed sale must be committed atomically with its inventory outbox record.

4.2 Major Runtime Components

Component	Deployment	Responsibility
React Web	Browser UI	Checkout, administration, reporting, and operational monitoring.
Kaching API	Cloud Run service	Authentication, authorization, business rules, REST API, sale finalization, reporting, and integration coordination.
Background Worker	Cloud Run worker service or worker pool	Outbox publication, message retry, card-payment reconciliation, and scheduled operational checks.
Peripheral Gateway	Node.js service on each POS terminal	Secure local access to card machine, barcode scanner, and receipt printer.
PostgreSQL	Cloud SQL	Authoritative transactional data, audit records, and outbox.
RabbitMQ	Managed quorum cluster	Durable at-least-once delivery of inventory events.

4.3 Layering and Dependency Rules

- Presentation code may call application services only through typed API clients.
- Express route handlers perform transport concerns and delegate business work to application services.
- Domain services contain rules and state transitions without depending directly on Express or RabbitMQ.

- Repositories isolate Prisma persistence operations.
- Integration adapters isolate K-Bank, terminal, printer, and RabbitMQ-specific contracts.
- Modules may exchange identifiers and documented interfaces, but must not query another module's tables directly through ad hoc SQL.

4.4 Suggested Backend Modules

Module	Core responsibilities
identity	Authentication, users, roles, store assignments, and session management.
organization	Stores, terminals, terminal registration, and operational context.
catalog	Global products, barcodes, prices, VAT configuration, and product search.
sales	Open carts, sale items, totals, discounts, cancellation, and sale state.
payments	Cash, card attempts, PSP adapters, uncertain outcomes, and reconciliation.
receipts	Receipt payload generation, print attempts, and reprints.
inventoryIntegration	Outbox records, RabbitMQ publication, retry, and status.
reporting	Store-level and consolidated daily summaries.
operations	Exception queues, system-health information, and recovery actions.
audit	Append-only security and business audit events.

5. Deployment Architecture

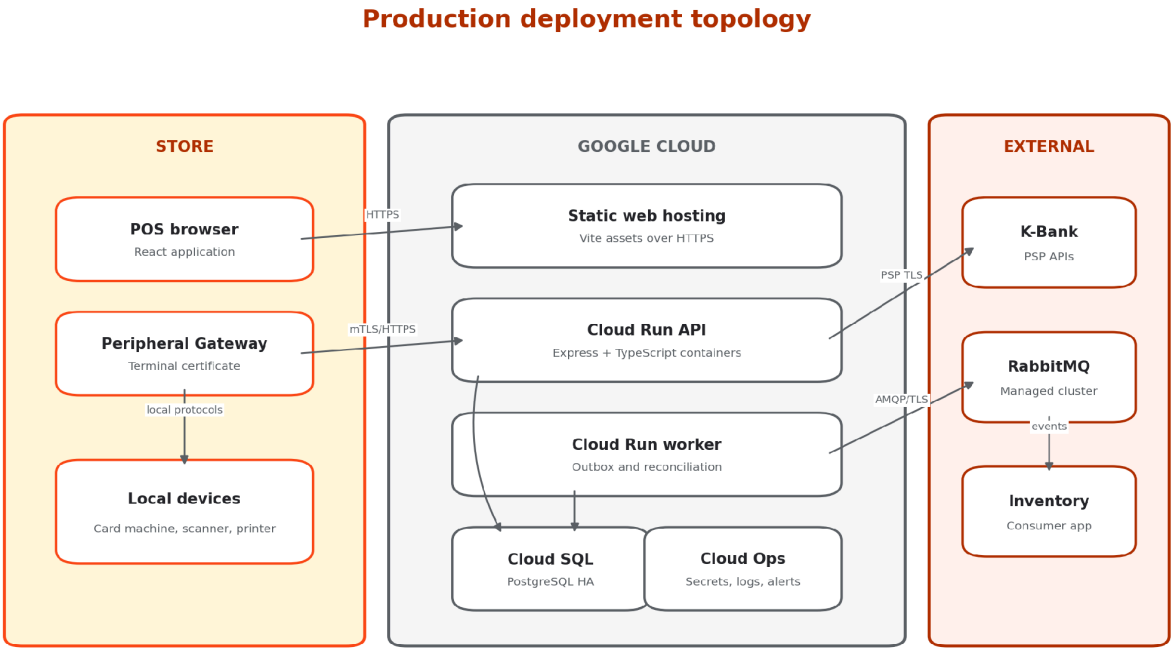

Figure 2. Production deployment topology

5.1 Production Topology

- The Vite production build is served as static assets over HTTPS using Google Cloud static hosting and CDN capabilities.
- The Express API runs in stateless containers on Cloud Run and scales horizontally.
- The background worker runs separately so message publication and reconciliation cannot consume checkout request capacity.
- Cloud SQL for PostgreSQL uses regional high availability, automated backups, and point-in-time recovery.
- RabbitMQ runs as a managed, TLS-protected, three-node quorum cluster in or near the selected GCP region.
- Each POS terminal runs a registered Peripheral Gateway with a terminal-specific certificate and outbound network access only.

5.2 Environment Separation

Environment	Purpose	Data policy
Development	Local implementation and automated tests using Docker Compose.	Synthetic data only.
Staging	Production-like integration, acceptance, load, and recovery testing.	Masked or synthetic data only.
Production	Live store operations.	Production data with restricted access and backup controls.

5.3 Connectivity Behavior

Kaching does not support offline checkout. If a terminal loses connectivity to the API, the web application shall disable sale start and completion actions, preserve any already displayed cart in memory for operator reference, and show a clear reconnect status. The cart is not treated as durable until the backend confirms it.

6. Application Structure

6.1 Frontend Structure

Area	Design
Framework	React with TypeScript and Vite.
Routing	Role-aware routes for checkout, catalog, reports, operations, and administration.
State	Server state through a query/cache layer; local component state for transient UI input.
Forms	Schema-based client validation matched to API contracts.
Money	API monetary values are decimal strings; the UI formats them as THB and never uses binary floating-point for financial calculations.
Errors	Inline validation for correctable input and persistent banners/dialogs for transaction-state or integration errors.
Accessibility	Keyboard navigation, visible focus, touch targets of at least 44 by 44 CSS pixels, and announced status changes.

6.2 REST API Convention

- Base path: /api/v1.
- JSON property names: camelCase.
- Timestamps: ISO 8601 in UTC; business-date interpretation uses the store timezone.

- Money: decimal strings with two fractional digits, for example "1250.00".
- Errors: a consistent problem-detail object containing code, title, message, fieldErrors, correlationId, and retryable.
- Idempotency: Idempotency-Key is mandatory for payment-attempt creation and sale finalization.
- Concurrency: mutable aggregate requests include a version value; stale updates return HTTP 409.
- Documentation: OpenAPI 3.1 is generated and checked in CI.

6.3 API Resource Outline

Resource area	Representative endpoints
Authentication	POST /auth/login, POST /auth/refresh, POST /auth/logout, GET /me
Sales	POST /sales, GET /sales/{id}, PATCH /sales/{id}/items, POST /sales/{id}/cancel, POST /sales/{id}/finalize
Payments	POST /sales/{id}/payment-attempts, POST /payment-attempts/{id}/terminal-result, GET /payment-attempts/{id}
Catalog	GET /products, POST /products, PATCH /products/{id}
Reports	GET /reports/daily-sales?date=&storeId=, GET /reports/daily-sales/consolidated?date=
Operations	GET /operations/payment-recovery, GET /operations/inventory-outbox
Administration	Users, role assignments, stores, and terminals under /admin.

7. Data Architecture

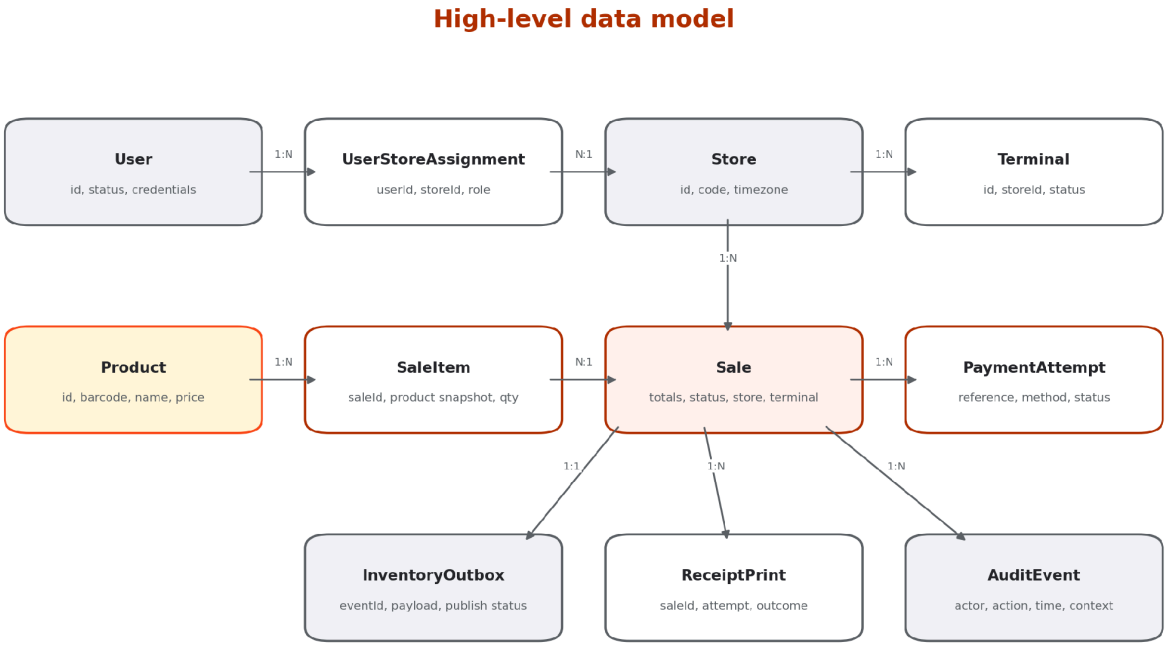

Figure 3. High-level data model

7.1 Core Entity Design

Entity	Key design notes
User	Authentication identity, active status, and security metadata. Roles and store scope are assigned separately.
Store	Stable code, name, active status, and Asia/Bangkok timezone.
Terminal	Belongs to one store and has a unique registration identity and active status.
Product	Global SKU, barcode, name, VAT rate or category, VAT-inclusive price, and active status.
Sale	Authoritative transaction aggregate with subtotal, discount, VAT, total, state, store, terminal, cashier, timestamps, and version.
SaleItem	Snapshots SKU, name, unit price, VAT information, quantity, and line amount so history is unaffected by later catalog changes.
PaymentAttempt	Unique merchant reference, method, requested amount, PSP reference, status, timestamps, and reconciliation metadata.
InventoryOutbox	Unique event ID, sale ID, event version, payload, state, attempt count, next attempt time, and last error.
ReceiptPrint	Print and reprint attempts, terminal, user, outcome, and error details without card data.
AuditEvent	Append-only actor, action, target, store, terminal, timestamp, correlation ID, and non-sensitive details.

7.2 Identifier and Monetary Strategy

- Use UUID primary keys for distributed uniqueness across stores and terminals.
- Optionally expose a human-readable sale number containing store code, business date, terminal code, and sequence, while retaining UUID as the technical key.
- Use PostgreSQL NUMERIC and Prisma Decimal for money. JavaScript number is not used for persisted financial calculations.
- Round percentage-derived discounts to two decimal places using commercial half-up rounding unless Thai statutory tax guidance requires a different rule.
- Persist the VAT rate and calculated VAT portion used by the sale so historical receipts remain reproducible.

7.3 Sale State Model

State	Meaning	Allowed next states
OPEN	Cart may be edited or cancelled.	PAYMENT_PENDING, CANCELLED
PAYMENT_PENDING	A cash confirmation or card attempt is in progress.	OPEN, PAYMENT_RECOVERY_PENDING, COMPLETED, CANCELLED where safe
PAYMENT_RECOVERY_PENDING	Card result is uncertain or approved payment cannot be finalized.	PAYMENT_PENDING, COMPLETED
COMPLETED	Payment succeeded and sale plus outbox were committed atomically.	None
CANCELLED	Sale was cancelled before completion.	None

7.4 Transaction Boundaries

Sale finalization transaction: In one PostgreSQL transaction, validate the current aggregate version and payment state, mark the sale COMPLETED, insert the inventory outbox record, and append required audit data. A failure rolls back the complete operation.

The system creates and commits a pending card PaymentAttempt before it sends any request to the terminal or PSP. This durable reference supports recovery when the external result arrives but local sale finalization fails.

8. Integration Design

8.1 K-Bank Card Payment

The production integration is isolated behind PaymentTerminalAdapter and PaymentProviderAdapter interfaces. Exact message formats, certificates, endpoints, terminal protocols, timeout values, and settlement procedures must follow the contracted K-Bank specification and certification process. The SDS does not invent provider-specific fields.

1. API creates a durable PaymentAttempt with a unique merchant reference.
2. The web application sends a signed payment instruction to the registered local Peripheral Gateway.
3. The gateway sends amount and reference to the card machine through the approved ECR protocol.
4. The gateway returns the terminal result and provider reference to the API over an authenticated channel.
5. The API records the result and finalizes an approved sale atomically with the inventory outbox.
6. On timeout or disconnection, the attempt becomes unresolved and the worker queries K-Bank using the original merchant reference.
7. No new card attempt or cash payment is permitted until the uncertain result is resolved.

8.2 Peripheral Gateway

- Runs as a managed Node.js and TypeScript service on each registered POS terminal.
- Exposes only a loopback TLS endpoint to the local browser and validates short-lived signed instructions from the API.
- Uses a terminal-specific certificate and refuses instructions for another terminal or store.
- Implements adapters for the K-Bank ECR protocol and ESC/POS-compatible printer driver.
- Never logs PAN, track data, PIN data, card verification values, or full provider payloads containing sensitive data.
- Reports health and device status without allowing arbitrary commands from the browser.

8.3 Receipt Printing

After completion, the API produces a receipt view model from the immutable sale snapshot. The web application sends a signed print job to the Peripheral Gateway. A print failure creates a failed ReceiptPrint record but never changes the sale from COMPLETED. Reprints require authorization and are logged.

8.4 Inventory Messaging

The inventory integration uses a transactional outbox and at-least-once RabbitMQ delivery. The Inventory Service must process messages idempotently using eventId or saleId plus event type.

Element	Design
Exchange	kaching.inventory, durable topic exchange.
Routing key	sale.completed.v1
Queue	inventory.sale.completed.v1, durable quorum queue.
Message	Persistent JSON message containing schemaVersion, eventId, saleId, storeId, terminalId, occurredAt, and item quantities.
Publishing	Worker uses mandatory routing and asynchronous publisher confirms.
Retry	Exponential backoff with jitter; PostgreSQL retains the outbox row until broker confirmation.
Consumer failure	RabbitMQ retains or redelivers unacknowledged messages.
Poison message	Rejected non-retryable messages move to a dead-letter queue and create an operational alert.

8.5 Failure-Handling Matrix

Failure	Required behavior	Operator visibility
Backend unreachable	Do not start or complete sales. Show reconnect status.	Checkout banner
PSP unavailable before request	Disable card option; allow cash or cancellation.	Checkout message
Card response unknown	Block further payment; reconcile original reference.	Recovery-pending state
Approved card, finalization failed	Do not charge again; reconcile and finalize from durable attempt.	Payment recovery queue
Printer failure	Keep sale completed; permit authorized reprint.	Print failure message and audit
RabbitMQ unavailable	Keep outbox pending and retry.	Inventory exception queue
Inventory consumer unavailable	RabbitMQ retains durable message until consumption resumes.	Broker monitoring
Outbox message rejected	Move to dead-letter handling and alert operations.	Failed inventory message

9. Security Design

9.1 Authentication and Session Security

- Passwords are hashed using Argon2id with current organization-approved parameters.
- Short-lived access tokens and rotating refresh tokens are stored in Secure, HttpOnly, SameSite cookies.

- State-changing requests use CSRF protection and server-side authorization checks.
- Disabled users, stores, or terminals cannot create sessions or submit transactions.
- Secrets and certificates are stored in Google Secret Manager or the secured terminal certificate store, never in source control.

9.2 Authorization

Role	System-wide access summary
Cashier	Checkout, permitted discounts, cash and card payments, and receipt printing.
Manager	Cashier functions, assigned-store daily summary, and authorized catalog maintenance.
Administrator	User, role, store, terminal, catalog, cross-store reporting, and operational exception access.
Head-Office Manager	Selected-store and consolidated reporting; operational access if separately granted.

9.3 Payment-Data Controls

Card-data boundary: Kaching stores only merchant reference, PSP reference, approved or declined status, amount, masked display data when contractually permitted, and timestamps. It does not store PAN, track data, PIN/PIN block, or card verification values.

The terminal, Peripheral Gateway, network path, and software involved in payment processing must follow the acquirer's certification requirements and the applicable PCI DSS scope. Sensitive authentication data must not be stored after authorization.[5]

9.4 Audit and Privacy

- Audit events are append-only and include actor, role, store, terminal, action, target, time, correlation ID, and outcome.
- Audit details exclude passwords, tokens, card data, and unnecessary personal data.
- Access to audit and operational data is itself authorized and logged.
- Production logs use structured redaction rules before export to centralized logging.

10. User Interface Standards

10.1 Theme and Visual Tokens

The official KMUTT colors for websites and applications are Orange #FA4616, Yellow #FFC72C, and Blue Grey #7B8189.[1] Kaching uses these brand colors with derived accessible neutrals and a darker orange for white-text controls.

Token	Value	Usage
brand.orange	#FA4616	Brand accents, selected indicators, large headings, and icons.
brand.yellow	#FFC72C	Highlights and warning surfaces with dark text.
brand.blueGrey	#7B8189	Borders, secondary accents, and large secondary text.
action.primary	#B33100	Primary button background with white text; derived for AA contrast.
text.primary	#24272A	Normal body text.
surface.default	#FFFFFF	Cards, forms, and primary work surfaces.
surface.subtle	#F2F4F5	Secondary panels and table striping.
status.success	#1F7A4D	Successful payment and completed sale status.
status.error	#B42318	Declines, failures, and blocked actions.

10.2 Checkout Layout Convention

- Use a two-region layout: product and cart workspace on the left, totals and action panel on the right.
- Keep subtotal, discount fields, VAT, and final total continuously visible.
- Place the final total in the strongest visual hierarchy and do not communicate state by color alone.
- Use large touch targets, visible keyboard focus, and scanner-friendly default focus behavior.
- Require confirmation for destructive actions such as cancelling a sale or deactivating a terminal.
- Use plain English and persistent recovery messages for uncertain card outcomes; never invite the cashier to retry while status is unknown.

10.3 Product Entry

The checkout screen accepts product entry through a keyboard-wedge USB barcode scanner, manual barcode entry, or product search by code or name. All three paths resolve the same global product record and current global selling price.

11. Observability and Operations

11.1 Structured Logging

Every API request and worker job receives a correlation ID. Logs are structured JSON and include service, environment, correlationId, saleId where applicable, terminalId, storeId, severity, event name, duration, and sanitized error details.

11.2 Metrics and Alerts

Signal	Example alert
API availability and latency	Error rate or p95 latency exceeds the defined threshold.
Payment recovery backlog	Any unresolved payment older than the operational threshold.
Outbox backlog	Pending count or oldest-message age exceeds threshold.
RabbitMQ health	Broker unavailable, quorum degraded, publish nack, or unroutable message.
Dead-letter queue	Any new inventory message enters the DLQ.
Database	Connection exhaustion, replication issue, storage growth, or failed backup.
Terminal gateway	Registered terminal has not reported health within threshold.

11.3 Operational Work Queues

Feature-P provides separate views for payment-recovery items and inventory-publication items. Operators may inspect status and retry eligible technical operations. They may not manually force a card result or alter a completed sale.

12. Performance, Availability, and Recovery

12.1 Performance Design

- Perform cart arithmetic in the browser for immediate feedback and revalidate all values on the server.
- Index Sale by storeId, completedAt, status, and payment method for daily summaries.
- Index Product by normalized barcode, SKU, active status, and searchable name fields.
- Use bounded database connection pools sized for Cloud Run scaling limits.
- Keep report queries read-only and paginated where detailed data is exposed.
- Load-test at 200 concurrent users and 20 completed sales per second, including outbox writes.

12.2 Availability Behavior

Dependency	Kaching behavior when unavailable
Kaching API or Cloud SQL	No new sale may start or complete.
K-Bank PSP	Card payment unavailable; cash remains available if the Kaching backend is healthy.
RabbitMQ	Sales continue because the outbox remains durable in PostgreSQL.
Inventory Service	Sales continue; RabbitMQ retains messages until the consumer recovers.

Dependency	Kaching behavior when unavailable
Receipt printer	Sales continue and remain completed; reprint is available later.

12.3 Backup and Disaster Recovery

- Use Cloud SQL automated backups and point-in-time recovery with settings capable of meeting the 15-minute RPO.[3]
- Use regional high availability for Cloud SQL and test failover behavior at least annually.
- Retain infrastructure definitions, container images, Prisma migrations, and configuration outside the database so services can be recreated.
- Document a recovery runbook that restores PostgreSQL, redeploys API and worker revisions, validates RabbitMQ connectivity, and checks payment and outbox backlogs.
- Perform a scheduled recovery exercise and record whether the 4-hour RTO and 15-minute RPO were met.

13. Development and Delivery

13.1 Repository Structure

Path	Contents
apps/web	React and Vite frontend.
apps/api	Express API application.
apps/worker	Outbox and payment-reconciliation worker.
apps/peripheral-gateway	Terminal-side Node.js service.
packages/contracts	Shared TypeScript schemas, API models, and event contracts.
packages/ui	Reusable Kaching UI components and KMUTT theme tokens.
packages/config	Shared linting, TypeScript, test, and build configuration.
prisma	Schema, migrations, and controlled seed data.
infra	Docker Compose and infrastructure-as-code definitions.

13.2 CI/CD Pipeline

1. Install dependencies using a locked package manifest.
2. Run formatting, linting, TypeScript compilation, unit tests, integration tests, and dependency/security checks.

3. Build versioned containers and publish them to Artifact Registry.
4. Deploy automatically to development, then promote the same images to staging and production after approval.
5. Run backward-compatible Prisma migrations before directing traffic to the new application revision.
6. Use Cloud Run revision rollback for application failures; database changes require an explicit forward-fix or tested rollback plan.

13.3 Configuration

Environment-specific values include database connection, RabbitMQ endpoints, PSP credentials, terminal certificate authority, receipt settings, retry thresholds, VAT configuration, logging level, and allowed frontend origins. Secrets are injected at runtime and never committed to the repository.

14. System Test Strategy

Test level	System-level focus
Unit	VAT, discount rounding, totals, permissions, state transitions, and retry policy.
Component	Each module with PostgreSQL or adapter substitutes.
Integration	Prisma transactions, RabbitMQ confirms, PSP contract, Peripheral Gateway, and printer protocol.
Contract	OpenAPI requests/responses and versioned inventory-event schema.
End-to-end	Cash sale, approved card, decline, PSP outage, unknown payment recovery, print failure, and reporting.
Fault injection	Database rollback, API restart, RabbitMQ outage, duplicate delivery, and worker crash after publish but before outbox update.
Performance	200 concurrent users and 20 completed sales per second while meeting latency target.
Security	Authorization scope, session controls, dependency scanning, secret leakage, and card-data logging checks.
Recovery	Cloud SQL restore, service redeployment, payment reconciliation, and outbox backlog recovery within RTO/RPO.
Accessibility	Keyboard checkout, focus order, status announcement, minimum resolution, and WCAG contrast.

15. Feature Allocation

Feature	Name	Primary design allocation
Feature-A	User Authentication and Access Control	Web, identity module, audit
Feature-B	Store and Terminal Operating Context	Web, organization module, Peripheral Gateway

Feature	Name	Primary design allocation
Feature-C	Product Catalog and Price Management	Web, catalog module, PostgreSQL
Feature-D	Sale Lifecycle Management	Web, sales module
Feature-E	Cart and Sale-Item Management	Web, sales module, catalog module
Feature-F	VAT-Inclusive Pricing and Total Calculation	Web, sales domain services
Feature-G	Order-Level Discount Management	Web, sales domain services, audit
Feature-H	Payment Method Selection	Web, payments module
Feature-I	Cash Payment Processing	Web, payments and sales modules
Feature-J	Card Payment Authorization	Web, payments module, Peripheral Gateway, K-Bank adapters
Feature-K	Card Payment Recovery and Reconciliation	Payments module, worker, K-Bank adapter, operations UI
Feature-L	Sale Finalization and Durable Recording	Sales module, Prisma transaction, PostgreSQL
Feature-M	Receipt Printing and Reprinting	Receipts module, web, Peripheral Gateway, printer
Feature-N	Inventory Update Outbox	Sales and inventoryIntegration modules, PostgreSQL
Feature-O	Inventory Message Delivery	Worker, RabbitMQ adapter
Feature-P	Operational Exception Monitoring	Operations UI and module, worker status data
Feature-Q	Daily Sales Reporting	Reporting UI and module, PostgreSQL
Feature-R	Audit Trail	Audit module and PostgreSQL
Feature-S	Administration and Master Configuration	Admin UI, identity and organization modules

16. Key Design Decisions and Risks

ID	Decision or risk	Treatment
ADR-01	Use a modular monolith rather than microservices.	Reduces deployment and transaction complexity while preserving module boundaries.
ADR-02	Use PostgreSQL transactional outbox plus RabbitMQ.	Prevents a completed sale from existing without a durable inventory update.
ADR-03	Use a terminal-side Peripheral Gateway for production hardware.	Keeps browser UI while supporting certified local protocols and silent receipt printing.
ADR-04	No offline checkout.	Avoids local data synchronization and payment conflict complexity; stores depend on backend connectivity.
ADR-05	Global catalog and global prices.	Simplifies reporting and administration; all stores see price changes after cache invalidation.
RISK-01	K-Bank ECR and PSP contracts may differ from assumptions.	Keep provider adapter interfaces and finalize details only after bank documentation and certification.
RISK-02	Peripheral Gateway increases endpoint operational burden.	Signed installers, auto-update, health checks, terminal certificates, and restricted commands.

ID	Decision or risk	Treatment
RISK-03	At-least-once delivery may duplicate events.	Stable event IDs, publisher confirms, and idempotent Inventory Service consumption.
RISK-04	Cloud backend outage stops checkout.	HA database, autoscaled API, monitoring, documented recovery, and clear store communication.

17. Required Upstream SRS Amendments

Traceability warning: The decisions approved after the original traceability matrix introduce behavior not yet numbered in the SRS. Add the following draft requirements before baselining this SDS and regenerate the requirements-to-features matrix.

Proposed ID	Proposed requirement	Feature
FR-039	The system shall allow an Administrator to create and deactivate user accounts.	Feature-S
FR-040	The system shall allow an Administrator to assign roles and permitted stores to users.	Feature-S / Feature-A
FR-041	The system shall allow an Administrator to create, update, and deactivate stores.	Feature-S / Feature-B
FR-042	The system shall allow an Administrator to register, activate, and deactivate POS terminals for a store.	Feature-S / Feature-B
FR-043	The system shall allow a cashier to add a product by barcode scan, manual barcode entry, or search by product code or name.	Feature-E
BR-021	The product catalog and selling price of each product shall be global and shall apply to all stores.	Feature-C
NFR-012	A terminal shall require connectivity to the Kaching backend to start or complete a sale; offline sales are not supported.	Feature-D / system constraint
NFR-013	The system shall support 20 stores, 10 active terminals per store, 200 concurrent users, and at least 20 completed sales per second.	Cross-cutting
NFR-014	The Kaching backend shall provide 99.5 percent monthly availability.	Cross-cutting
NFR-015	The production service shall meet an RPO of 15 minutes and an RTO of 4 hours.	Cross-cutting
NFR-016	The English-only user interface shall support desktop and touchscreen use at 1280 x 720 or higher and meet WCAG 2.1 AA color contrast.	Cross-cutting UI

18. References

[1] KMUTT Corporate Identity: official website and application colors.

[2] Google Cloud Run overview: managed container application platform.

[3] Google Cloud SQL for PostgreSQL documentation and backup/recovery guidance.

[4] RabbitMQ documentation: publisher confirms, consumer acknowledgements, and durable queues.

[5] PCI Security Standards Council guidance on storage of sensitive authentication data.

End of document